

CS 3960 (3)

Vibe Coding

Spring 2026, *content* Pavel Panchekha and John Regehr, *illustrations* Gemini 3 Nano Banana Pro

- No new reading assignment today
- Please make sure you're working with the code belonging to the NEXT student in the uNID list, not the previous
- If you haven't checked that you can run the editor you're modifying (and its tests) do that today
- Submit a pull request by Friday 5pm
 - It should include additional tests
 - **How's this going so far?**

Writing code alone

#yolo #shipit

...with classmates

use version control

probably just commit
and live in the
moment

check over code and
test before
submitting

...with work team

???



John stole this slide

Let's talk about code review

At the most basic level it means:

Someone other than the code's
author reads and evaluates it

First, why do we do this?





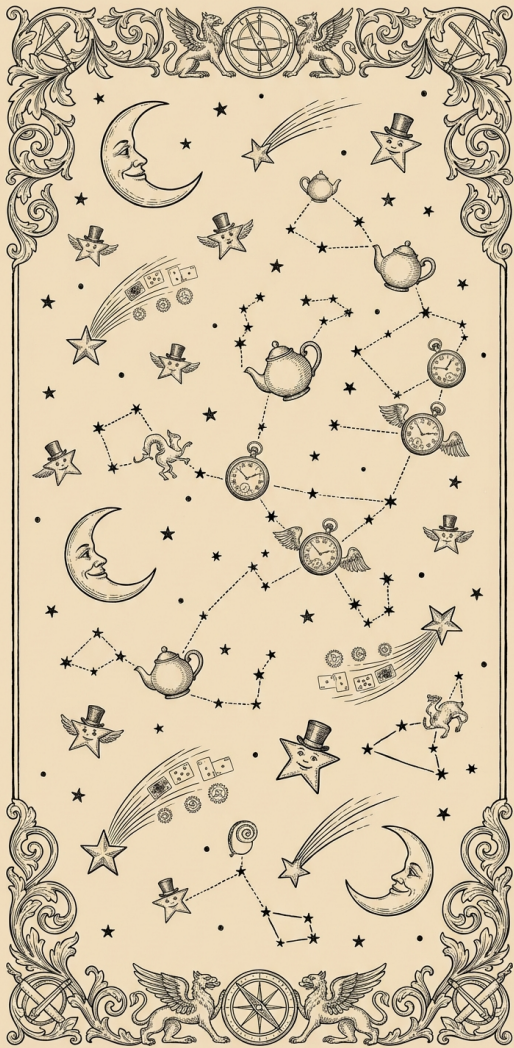
Code review is for...

- Detecting bugs
- Improving readability
- Improving maintainability
- Sharing knowledge
- Creating a shared sense of ownership

Code review is **not** just for pull requests

... but that's how we'll mainly use it in this class.





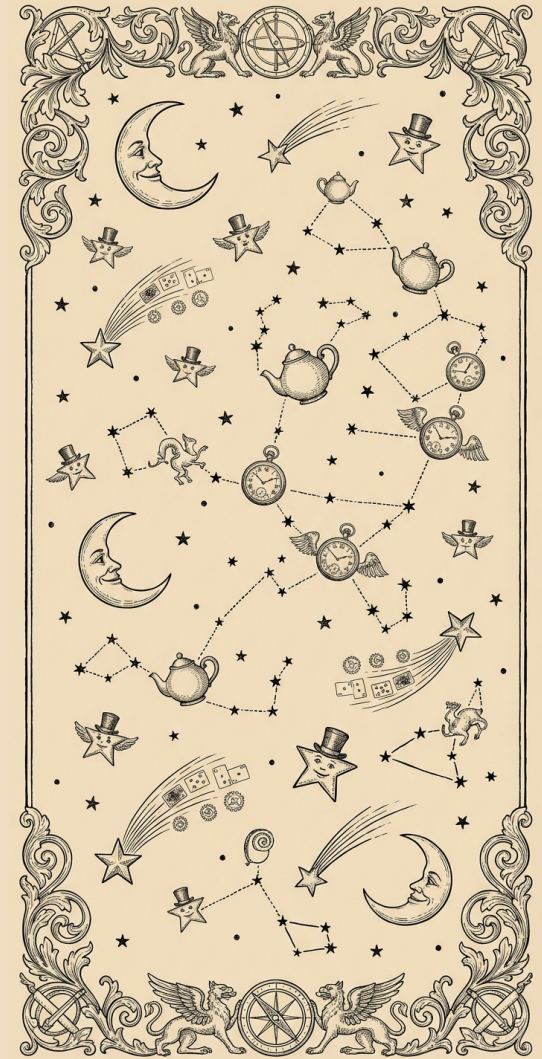
In a code review, do ask:

- Is the logic clear and robust?
- Are the tests and docs adequate?
- Can we keep building on this?

But don't:

- Hold up changes over nitpicks
- Insist that everything is micro-optimized

How does *AI* affect code
review?



Some code is hard to test

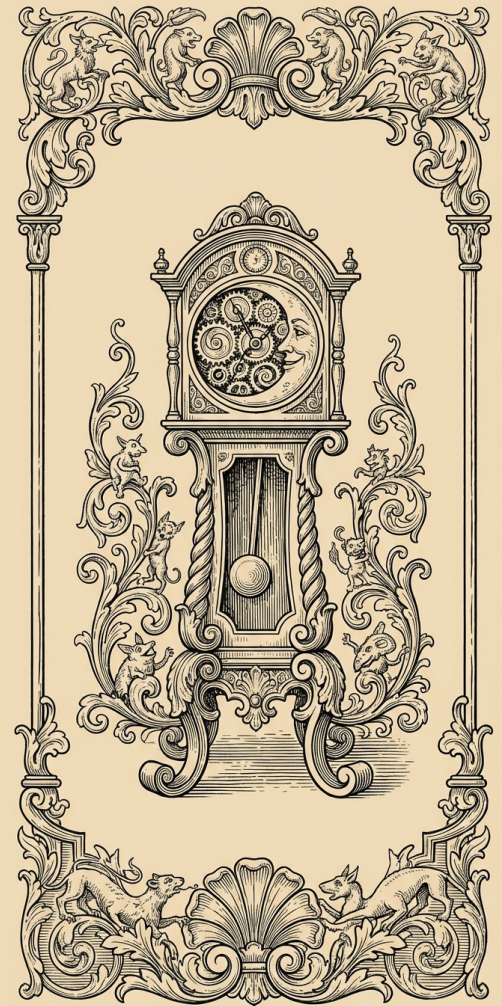
Examples?



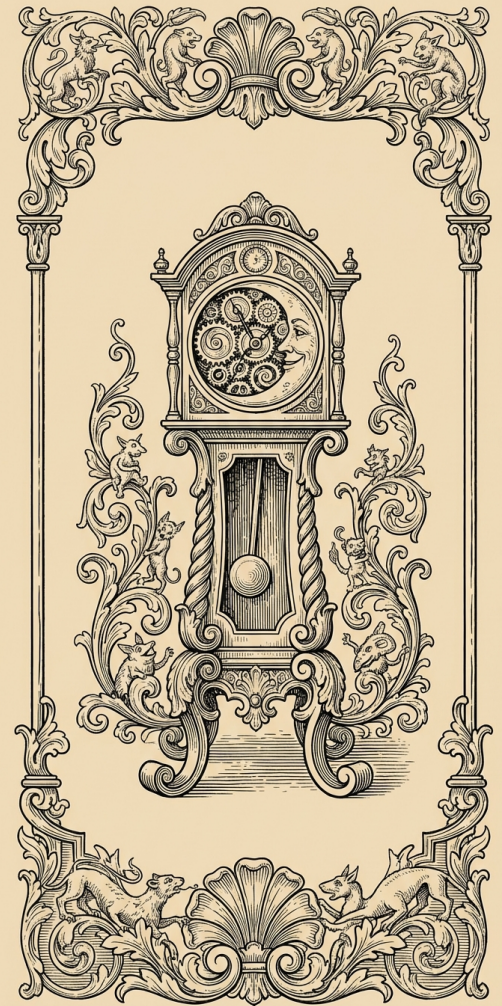
Code is hard to test when...

- It's code that you wrote
- It interacts with the world non-programmatically
 - E.g. GUIs, embedded systems, etc
- It is concurrent or otherwise non-deterministic
- It's a complex one-off system
 - E.g. Simulating tornado internals

```
void func(a, b, c) {  
    foo1(a);  
    tmp = getCurrentDayAndTime();  
    foo2(tmp, b);  
    foo3(tmp, c);  
}
```




```
void func(a, b, c, dayAndTime) {  
    foo1(a);  
    foo2(dayAndTime, b);  
    foo3(dayAndTime, c);  
}
```





How can we design software so that it's easier to test?

This is super important in practice, and the LLM is not going to do this stuff on its own.

Anything non-deterministic can be determinized

- Concurrency can be disabled
- PRNG seeds are externally controlled
- What else?





Components are not needlessly stateful

- Often we isolate state in a database, key-value store, etc.
- E.g. REST applications

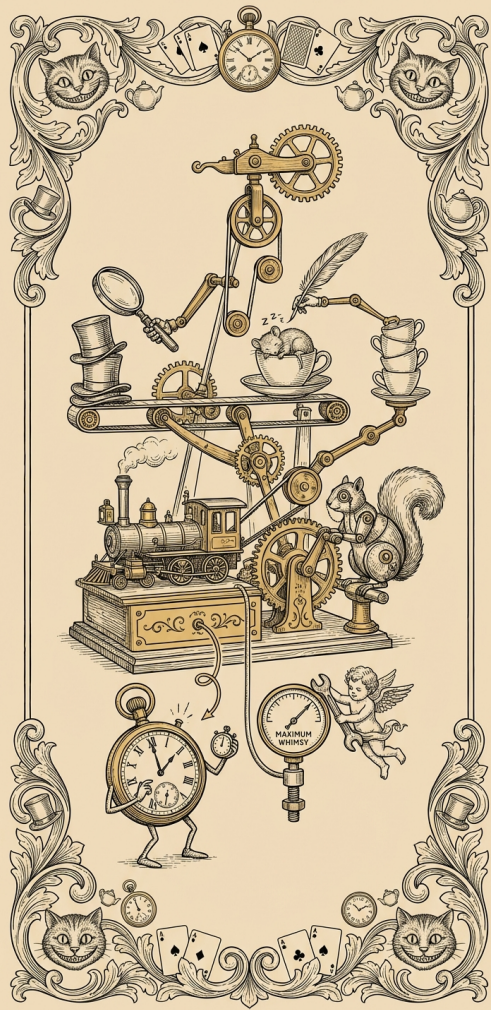
Components that are stateful expose an API for loading / saving / printing their state

All key interfaces are exposed
programmatically

I/O lives at the outside of the
modular structure

Tokens, keys, server addresses are
passed in as arguments — not
hard-coded





Optimizations can be disabled

- E.g. caching, memoization, fast paths

Concerns are separated

- E.g. don't run database queries from the user interface

Important expectations and behaviors are documented

External APIs have robust error checking

Preconditions, postconditions, and data invariants are checked using assertions





Program defensively:

Assume users will do crazy things

Assume the internet is hostile

Assume the network will go down, and
then come up, and then go down again

Assume the disk will fill up

What else?



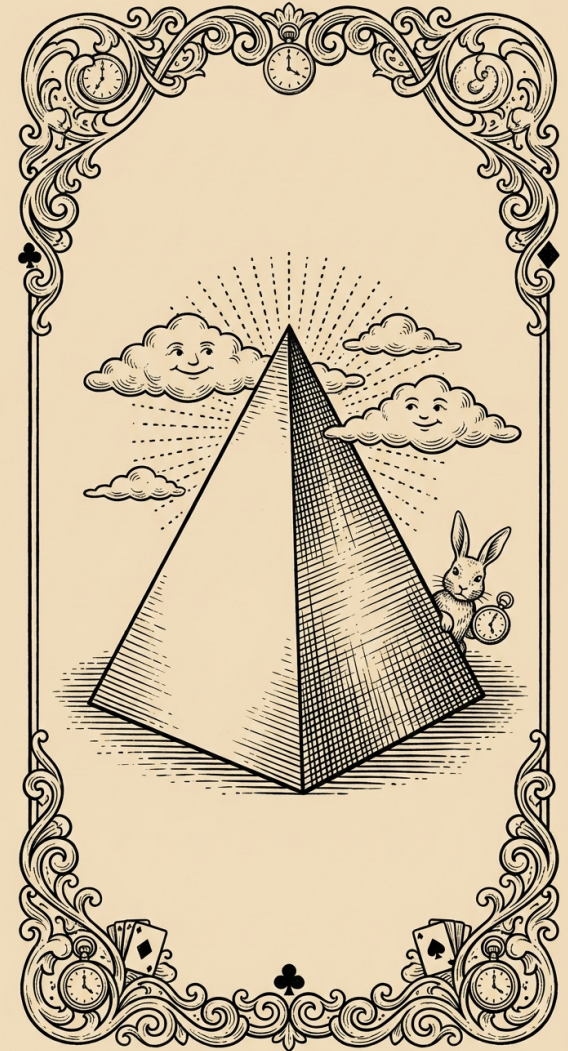
Defensive programming:

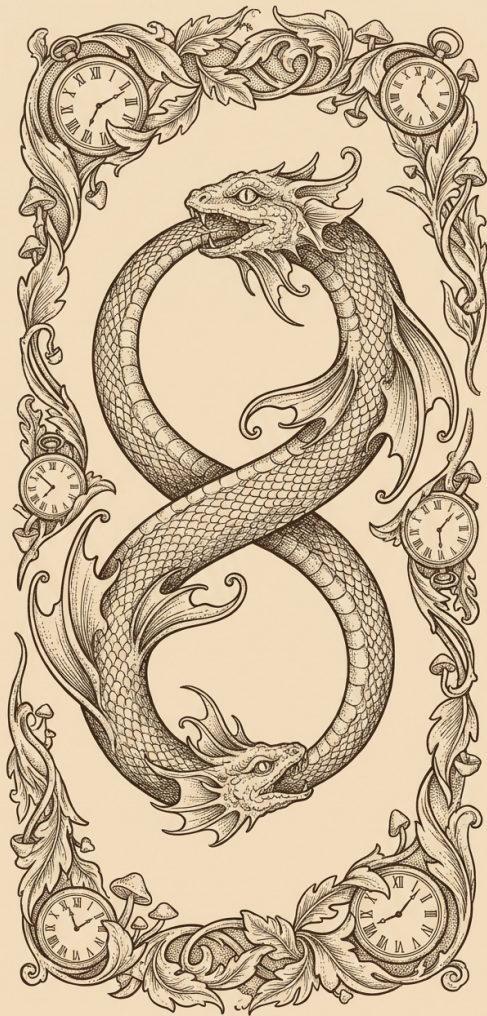
When something goes unexpectedly wrong, complain loudly (or crash)

When something goes expectedly wrong, either handle it carefully and gracefully, or else complain loudly (or crash)

Minimize the number of modes
in your software

Minimize the number of
configuration options in your
software



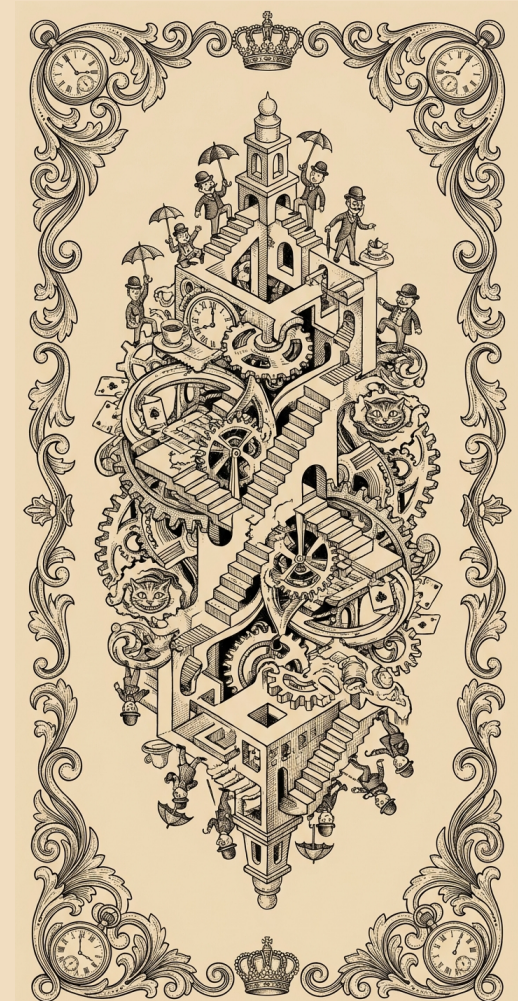


Write the code and its tests at the same time.

This way, if your code is hard to test, you'll find this out sooner than later.

There exist systems that are
effectively impossible to test.

Try to not create them.





Avoid test cases that...

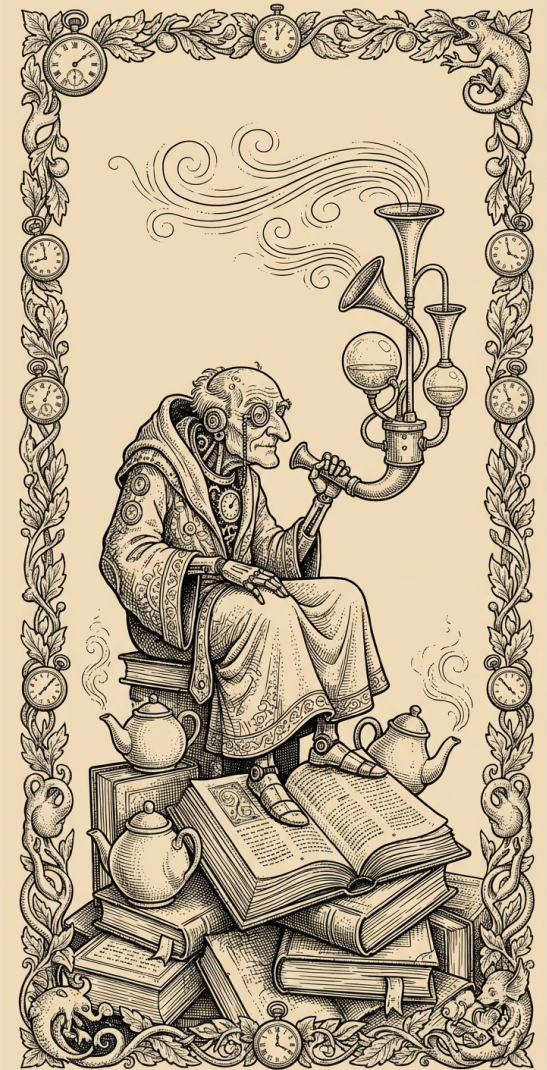
- Are not deterministic
- Are very slow to execute
- Don't have a specific purpose
- Don't elicit a behavior that we can agree is correct or incorrect

A **test oracle** tells us whether a test has passed or failed

The trivial oracle is a hard-coded answer

But many others are possible

Name a few

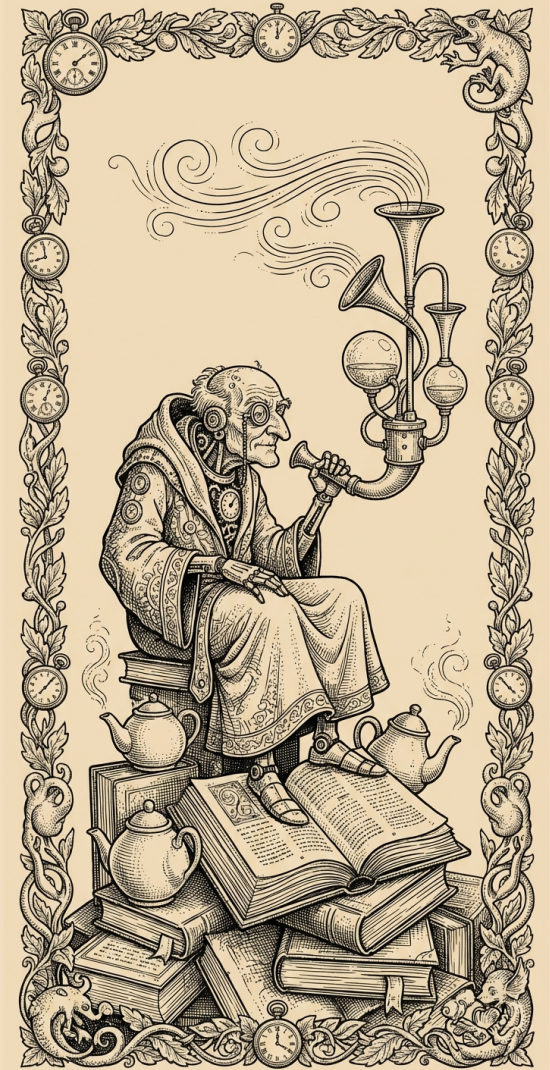


Some less obvious oracles...

- Language / environment rules
- Resource limits
- Internal consistency of the result
- Approximate bounds known by a human
- Differential — compare against a second implementation
- A specification — executable or otherwise

Creative and clever test oracles
can make or break a testing
effort

It's always worth asking: **What
kind of oracles could I use to
better test my software?**





Q: When do tests get run?

A: In general, the sooner the better.

The longer a bug lasts, the more expensive it is to fix.

Many project end up with a hierarchy of tests

- If you're modifying just one subsystem, then you start running the unit tests for that subsystem
- Then, the unit tests for the rest of the subsystems
- Finally, run integration tests

- Clone the calculator app in the in-class quiz linked to the Week 4 canvas module
- Follow the instructions!
- You can work alone or in a pair
- If you're in a pair, make sure to both submit the quiz answers!!!